

WC Bill Review - 10-Minute Talk Script

10-minute talk script

For presenting the one-slide summary ([WC_Bill_Review_One_Slide_Summary.pptx](#)) to the leadership team. The same script is embedded as speaker notes in that file — this is a standalone copy to read from, print, or rehearse with. Timings are approximate; talk at a natural pace rather than watching the clock.

[0:00-0:45] Open

“Today I want to walk you through a project I built end to end: an AI system that reviews workers’ comp medical bills and catches coding problems automatically. Everything you’ll see was built and tested on 100% synthetic data — no real patients, providers, or claims anywhere — but it’s checked against real, publicly published coding rules: CPT/HCPCS, ICD-10-CM, NCCI edit pairs, and CMS’s MUE tables. This is a status update on a finished piece of work, not a pitch for new resources.”

[0:45-2:15] The business issue (*left panel*)

“Start with the problem. When a workers’ comp bill comes in, someone has to check it against a large, constantly-changing rulebook — is this procedure pair allowed together, was this code billed too many times today, does the treatment even match the injury, was the visit coded at a level the documentation supports? Doing that by hand, line by line, on every bill doesn’t scale. And when a reviewer misses one, it’s not a rounding error — it’s either money paid out on a bill that should have been caught, or, if you overcorrect, reviewer time wasted chasing bills that are actually fine. Four problems recur most often: unbundling, fragmented billing, causality mismatches, and upcoding — and that’s exactly what this system targets.”

[2:15-4:00] The approach — the RAG process (*second panel*)

“So the approach: automate the first pass, but keep it grounded in the real rules rather than a model’s best guess. Four steps, and I want to slow down on the last two because they’re a real RAG pipeline, not just a label. Step one — score every bill against the actual coding-rule tables. Step two — confirm the violation deterministically; a plain rule-checker, never a guess. Step three — retrieve the exact rule text the bill broke, from a local vector store. Step four — only then does a language model generate the explanation, grounded in that retrieved text. That’s retrieval-augmented generation in practice: the model is never asked to recall coding rules from its own memory, it’s handed the specific, already-confirmed rule and asked to explain it in plain language.”

[4:00-6:00] What was built (*third panel*)

“That approach turned into five real pieces, and I want to be specific because this is what’s actually running today, not a plan. One — a synthetic bill generator that builds thousands of realistic bills against the real rule tables. Two — an XGBoost model that scores all four checks at once, not just a single flag. Three — a drift-monitoring pipeline, because these coding rules get revised quarterly and annually by CMS and AMA, so a model can go stale without warning; this includes a retraining trigger. Four — a retrieval-grounded explanation agent: a local Chroma vector store holds the rule text, and Azure OpenAI only writes the plain-language reason after the exact rule has been retrieved and confirmed. And five — it’s all wrapped in a FastAPI web service with a working chat UI, and packaged as a single Docker container that runs the same way on any machine. You give it one command and it’s up.”

[6:00-8:30] The result (*fourth panel*) — spend the most time here

This is the honest core of the talk — don’t rush it.

“Now the results, and I want to be upfront: the first evaluation I ran looked perfect — 1.0 precision and recall on everything — and that number is misleading, not impressive. Several features I engineered were literally restatements of the rule that generated the label, so of course the model found them. The real test is the ablation evaluation, where I stripped those out. And that’s what’s on the slide: unbundling and fragmented billing recover strongly — 97 and 100 percent precision. Upcoding stays

perfect because it has genuinely independent documentation fields behind it. Causality mismatch collapses — 50 percent precision, 3 percent recall — and that’s the honest finding of this whole project. Without a direct rule feature, the model doesn’t have enough independent signal yet. I know exactly why, and I know the fix — add the diagnosis’s own body region as a feature — I just haven’t built it yet, and I’d rather tell you that than hide it. At the operating threshold I’d recommend as a starting point, about 9 in 10 flags are real and it catches roughly three-quarters of violations, because in this domain a missed violation costs more than a reviewer double-checking a clean bill. And separately, the drift monitor: I simulated a real-world pattern — claims staying open longer — and the system correctly detected it and recommended retraining, including being honest about one feature that was probably just sampling noise rather than real drift. One more honest data point, on the RAG side specifically: I ran it end to end for real — real retrieval, real generation — and every explanation it produced stayed grounded in the retrieved rule text and the bill’s own facts, no invented codes, no outside-knowledge claims. Unlike the oncology project I also built, this retrieval pipeline is actually wired into the live agent, not just validated separately.”

[8:30-9:40] Status & close

“Where this stands: weeks one through four are complete, plus a partial week six — the API and Docker packaging both work. What’s not done is explicit, not implied: a fine-tuned narrative generator was scoped and deliberately cut for cost reasons, and cloud deployment to Azure hasn’t happened — the container runs correctly locally, it just hasn’t been pushed and hosted yet. Every number I showed you today came from a real, reproducible run of this code, with fixed seeds — nothing here is an estimate.”

[9:40-10:00] Invite questions

“That’s the full arc — issue, approach, what’s built, and an honest read of the results. Happy to go deeper on any piece — the model evaluation, the drift monitoring, or the explanation agent.”

Delivery notes

- **Slowest section on purpose:** the results section (6:00-8:30) is the longest because it’s where the honesty of the project shows — don’t compress it to make room elsewhere.
- **If you’re running short on time,** trim the “what was built” section (4:00-6:00) down to naming the five pieces without elaborating on each — the slide itself carries that detail.
- **If you’re running long,** the open (0:00-0:45) and the close (9:40-10:00) are the safest places to tighten — the middle three sections are the substance.